

AI Study Companion — AI Tools Usage

AI Tools Usage

Two clearly separated concerns. Section 1 is about the tools used to **build** this product (process). Section 2 is about the AI used **within** the product itself (runtime features the user interacts with). Do not conflate the two.

1. AI tools used to BUILD this product

Tool	What it was used for
Claude (architecture + prompt generation)	Drafted <code>docs/architecture.md</code> (the single source of truth: schema, RAG flow, security model, event system) and the per-phase build prompts in <code>docs/build-prompts.md</code> that drove implementation order (phases 02-19).
OpenCode (coding agent)	Implemented every phase from those prompts: scaffolding, migrations, auth, services, RAG, Tutor/Quiz/Assessment/Mastery/Growth/Recommendations, analytics, admin, observability, tests/eval, deployment prep, docs. Also debugging (e.g. closing the phase-15 <code>EMBEDDING / CONCEPT_EXTRACTION</code> logging gaps, fixing the <code>RETRIEVAL_FIXTURES</code> circular-type build error) and writing the per-phase logs in <code>docs/development-prompts/</code> .
Muse Spark (model powering this session)	The large language model behind the OpenCode agent for the later phases (observability, evaluation suite, deployment, documentation, sidebar UI rework): code edits, test/fixture design, verification runs, and these documentation artifacts. Report issues at https://github.com/anomalyco/opencode .

What AI did **not** do: it did not provision Supabase/Vercel/Inngest accounts, did not run the live-Vercel smoke test (manual step in phase 18), and did not record the demo video. All verification claims in `docs/evaluation.md` and the phase logs come from actually executed commands (`npm test`, `npm run eval`, `npm run build`, live Supabase queries) — outputs were recorded, not hypothesized.

2. AI used WITHIN the product itself

All runtime AI goes through the thin `AIService` abstraction (`lib/ai/AIService.ts`). Provider split (verified live 2026-09-15): chat / structured generation / evaluation → Meta's Llama API (`https://api.llama.com/compat/v1`, model `Llama-4-Maverick-17B-128E-Instruct-FP8`, JSON mode via `response_format: { type: "json_object" }` + server-side schema validation); embeddings → Groq (`https://api.groq.com/openai/v1`, model

omic-embed-text-v1.5 , 768 dims) because Meta's Llama API has no embeddings endpoint (verified 404). `chunks.embedding VECTOR(768)` matches Groq output.

Feature	AI role	Human/deterministic guardrails
Grounded Tutor (TUTOR)	Answers questions from retrieved chunks; returns {answer, confidence, grounded, citations, followUpSuggestion}	Retrieval scoped by <code>project_id + user_id</code> with <code>RELEVANCE_THRESHOLD=0.25</code> ; insufficient evidence → fixed response, no LLM call; evidence wrapped in <code><retrieved_evidence></code> as untrusted data (prompt-injection tested); citations validated server-side; every call logged to <code>ai_operations</code> .
Quiz generation (QUIZ_GENERATION)	Generates N questions per selected concepts/difficulties/types	Concept selection is backend weighted scoring (mastery + mistakes + trend + recency/frequency); output validated, retried once on shape failure; duplicate-question check vs last quiz.
Open-ended evaluation (OPEN_ENDED_EVALUATION)	Grades free-text answers → {score 0-100, understanding, strengths, missingConcepts, reasoningQuality, feedback}	MCQ grading is deterministic (string compare, no LLM); open-ended output validated before persisting; failure raises instead of writing a guess.
Concept extraction (CONCEPT_EXTRACTION)	Extracts {name, description}[] from uploaded documents	Non-fatal: failure logs and yields [] so material never sticks in <code>PROCESSING</code> ; upsert linked via <code>source_material_id</code> .
Recommendations (RECOMMENDATION)	Produces {title, action_items} from weak concepts, mistakes, mastery, goal, activity	System prompt bans generic advice; validator enforces 2-5 specific items naming real concepts; skipped when nothing is weak (<code>mastery <60</code> or <code>REQUIRES_ATTENTION</code>).
Embeddings (EMBEDDING)	Chunk + query vectors for pgvector cosine search	<code>match_chunks</code> RPC filtered by <code>project_id</code> ; JS-cosine fallback for dev; exactly one <code>ai_operations</code> row per call incl. failures with <code>request_id</code> traceability.

Deterministic (never LLM-decided): mastery math (`new = prev×0.7 + evidence×0.3`), growth classification (`IMPROVING >+5` / `REQUIRES_ATTENTION <-5` / `else STABLE`), ownership checks (`WHERE id=$1 AND user_id=$2 + RLS`), and all analytics aggregations.

Cost/latency visibility: every call writes `ai_operations(feature, model, request_id, latency_ms, success, tokens_in/out, estimated_cost, error)`; surfaced in `/admin/ai-usage` and project/global analytics.